

DIGITAL LOGIC AND COMPUTER DESIGN

TEAM DLCD
Department of CSE and CSE (AI & ML)

DIGITAL LOGIC AND COMPUTER DESIGN

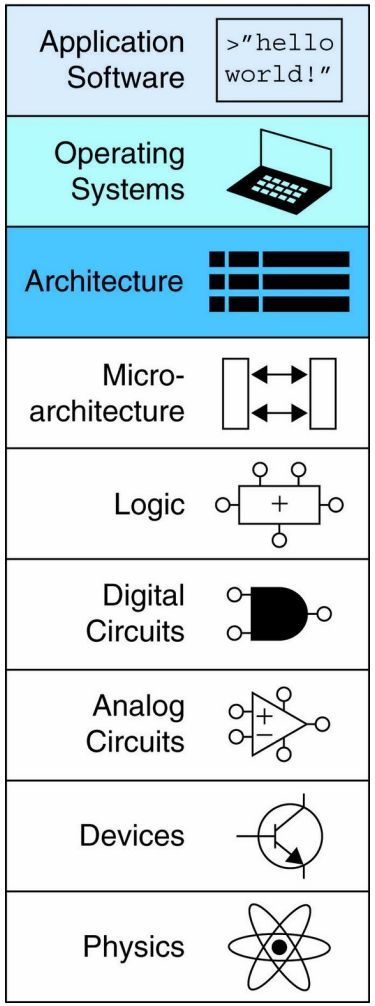
ARCHITECTURE-Programming

TEAM DDCO
Department of CSE and CSE (AI & ML)

DIGITAL LOGIC AND COMPUTER DESIGN

TOPICS

- Arithmetic/Logical Instructions
- Branching
- Conditional Statements.
- Getting Loopy
- Arrays
- Function Calls.



© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Programming

- High–level languages:
  - e.g., C, Java, Python
  - Written at higher level of abstraction
- Common high–level software constructs:
  - if/else statements for loops
  - while loops
  - Arrays, function calls
  - In this section, we explore how to translate these high-level constructs into MIPS assembly code.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Arithmetic/Logical Instructions

MIPS provides a variety of arithmetic and logical instructions that are used to implement higher-level programming constructs. Most arithmetic instructions operate on values stored in registers.

Arithmetic instructions

```
add  $t0, $s1, $s2      # $t0 = $s1 + $s2
sub  $t0, $t3, $t5      # $t0 = $t3 - $t5
addi $t0, $s3, -12      # $t0 = $s3 - 12
```

For example, if $s1 = 25 and $s2 = 10:

add $t0,$s1,$s2 ⇒ $t0=35

MIPS also provides separate instructions for **multiplication and division**, which use the special **hi and lo** registers.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Arithmetic/Logical Instructions

MIPS provides four basic logical operations: **AND, OR, XOR and NOR**. These are R-type instructions and operate **bit-by-bit** on two source registers.

For example, consider:

$s1 = 0xFFFF0000
$s2 = 0x46A1F0B7

The results are:

```
and $s3, $s1, $s2    # $s3 = 0x46A10000
or  $s4, $s1, $s2    # $s4 = 0xFFFFF0B7
xor $s5, $s1, $s2    # $s5 = 0xB95EF0B7
nor $s6, $s1, $s2    # $s6 = 0x00000F48
```

AND is useful for masking bits because it can force selected bits to 0. **OR can combine bits** from two values.

MIPS does not provide a separate NOT instruction. Instead,

NOT A=A NOR $0

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Logical Instructions: What to Remember

PES UNIVERSITY

AND
Mask bits (keep only selected 1s)

OR
Set / combine bits

XOR
Show differences between bits

NOR
OR first, then invert (NOR with $0 = NOT)

- andi, ori, and xori zero-extend the 16-bit immediate before the operation.
- Use **and for masking**,

  or for merging fields,

  xor for bit comparison,

  nor for inversion tricks.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Logical Instructions: Example 1

Source Registers

$s1	1111	1111	1111	1111	0000	0000	0000	0000
$s2	0100	0110	1010	0001	1111	0000	1011	0111

Assembly Code

```
and $s3, $s1, $s2
or  $s4, $s1, $s2
    $s5, $s1, $s2
    $s6, $s1, $s2
```

Result

$s3								
$s4								
$s5								
$s6								

xor

nor

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Logical Instructions: Example 1

Source Registers

$s1	1111	1111	1111	1111	0000	0000	0000	0000
$s2	0100	0110	1010	0001	1111	0000	1011	0111

Assembly Code

```
and $s3, $s1, $s2
or  $s4, $s1, $s2
xor $s5, $s1, $s2
nor $s6, $s1, $s2
```

Result

$s3	0100	0110	1010	0001	0000	0000	0000	0000
$s4	1111	1111	1111	1111	1111	0000	1011	0111
$s5	1011	1001	0101	1110	1111	0000	1011	0111
$s6	0000	0000	0000	0000	0000	1111	0100	1000

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Logical Instructions: Example 2

Assembly Code

```
andi $s2, $s1, 0xFA34
ori  $s3, $s1, 0xFA34
xori $s4, $s1, 0xFA34
```

Source Values

$s1	0000	0000	0000	0000	0000	0000	1111	1111
imm	0000	0000	0000	0000	1111	1010	0011	0100

zero-extended

Result

$s2								
$s3								
$s4								

- Logical operations can also operate on immediates. These I-type instructions are andi, ori, and xori.
- **nori is not provided, because the same functionality can be easily implemented using the other instructions.**
- Because these instructions operate on a 32-bit value from a register and a 16-bit immediate, they first zero-extend the immediate to 32 bits.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Logical Instructions: Example 2

Assembly Code

```
andi $s2, $s1, 0xFA34
ori  $s3, $s1, 0xFA34
xori $s4, $s1, 0xFA34
```

Source Values

$s1	0000	0000	0000	0000	0000	0000	1111	1111
imm	0000	0000	0000	0000	1111	1010	0011	0100

zero-extended

Result

$s2	0000	0000	0000	0000	0000	0000	0011	0100
$s3	0000	0000	0000	0000	1111	1010	1111	1111
$s4	0000	0000	0000	0000	1111	1010	1100	1011

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Think About It

- MIPS instruction set has an **or** instruction as well as a **nor** instruction, and also an **ori** instruction, but no **nori** instruction.
  - Why?
  - How can **nori** functionality be implemented?

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Think About It: Why is there no nori?

- MIPS avoids adding instructions whose behavior can already be built using existing ones.
- nori would mean: destination = NOT(rs OR immediate).
- That behavior is already easy to create in two instructions.

```
ori  $t0, $s1, 0x00FF
nor  $t0, $t0, $zero

Result:
$t0 = ~( $s1 | 0x00FF )
```

Key idea: nor with $zero works like bitwise NOT, so a dedicated nori instruction is unnecessary.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Shift Instructions

- Shift instructions shift the value in a register left or right by up to 31 bits.
- Shift operations multiply or divide by powers of two.
- MIPS shift operations are ***sll*** (shift left logical), ***srl*** (shift right logical), and ***sra*** (shift right arithmetic).

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Shift Instructions

- Left shifts always fill the least significant bits with 0's.
- However, right shifts can be either logical (0's shift into the most significant bits) or arithmetic (the sign bit shifts into the most significant bits).
- Figure shows the machine code for the R–type instructions sll, srl, and sra.
- rt (i.e., $s1) holds the 32–bit value to be shifted, and shamt gives the amount by which to shift (4).
- The shifted result is placed in rd.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Shift Instructions: Example

Assembly Code

```
sll $t0, $s1, 4

srl $s2, $s1, 4

sra $s3, $s1, 4
```

Field Values

op	rs	rt	rd	shamt	funct
0	0	17	8	4	0
0	0	17	18	4	2
0	0	17	19	4	3
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Machine Code

op	rs	rt	rd	shamt	funct	
000000	00000	10001	01000	00100	000000	(0x00114100)
000000	00000	10001	10010	00100	000010	(0x00119102)
000000	00000	10001	10011	00100	000011	(0x00119903)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Figure Shift instruction machine code

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Shift Instructions: Example

- Figure shows the register values for the shift instructions sll, srl, and sra.
- Shifting a value left by N is equivalent to multiplying it by 2^N.
- Likewise, arithmetically shifting a value right by N is equivalent to dividing it by 2^N,

	Source Values							
$s1	1111	0011	0000	0000	0000	0010	1010	1000
shamt								00100

Assembly Code		Result							
sll $t0, $s1, 4	$t0	0011	0000	0000	0000	0010	1010	1000	0000
srl $s2, $s1, 4	$s2	0000	1111	0011	0000	0000	0000	0010	1010
sra $s3, $s1, 4	$s3	1111	1111	0011	0000	0000	0000	0010	1010

Figure Shift operations

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Shift Instructions: sll vs srl vs sra

Instruction	What it does	Bits filled in	Typical use
sll	Shift left logical	0s on the right	Multiply by 2^n (if no overflow)
srl	Shift right logical	0s on the left	Unsigned values / bit extraction
sra	Shift right arithmetic	Copies sign bit	Signed divide by 2^n

```
Example value:
10010000

srl by 2 → 00100100
sra by 2 → 11100100
```

shamt stores the fixed shift amount (0–31).

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Variable Shift Instructions

- MIPS also has variable–shift instructions: sllv (shift left logical variable), srlv (shift right logical variable), and srav (shift right arithmetic variable).
- Figure shows the machine code for these instructions.
- Variable–shift assembly instructions are of the form sllv rd, rt, rs.
- The order of ***rt*** and ***rs*** is reversed from most R–type instructions.
- rt ($s1) holds the value to be shifted, and the five least significant bits of rs ($s2) give the amount to shift.
- The shifted result is placed in rd, as before.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Variable Shift Instructions: Example

Assembly Code / Field Values

Assembly Code	op	rs	rt	rd	shamt	funct
`sllv $s3, $s1, $s2`	0	18	17	19	0	4
`srlv $s4, $s1, $s2`	0	18	17	20	0	6
`srav $s5, $s1, $s2`	0	18	17	21	0	7
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Machine Code

op	rs	rt	rd	shamt	funct	
000000	10010	10001	10011	00000	000100	(0x02519804)
000000	10010	10001	10100	00000	000110	(0x0251A006)
000000	10010	10001	10101	00000	000111	(0x0251A807)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Figure Variable-shift instruction machine code

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Variable Shift Instructions: Example

- The shamt field is ignored and should be all 0's.
- Figure shows register values for each type of variable–shift instruction.

	Source Values							
$s1	1111	0011	0000	0100	0000	0010	1010	1000
$s2	0000	0000	0000	0000	0000	0000	0000	**1000**

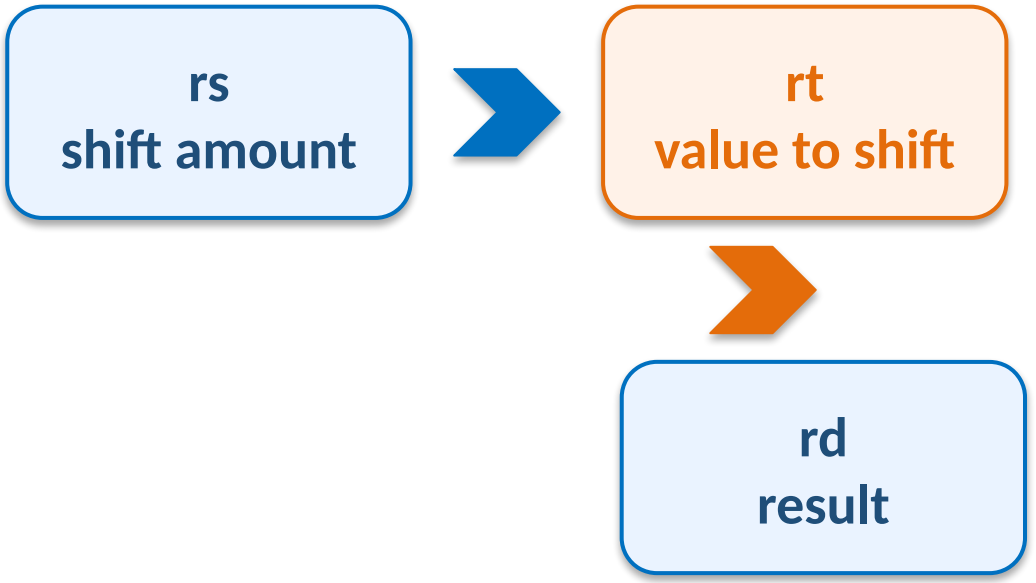
Assembly Code		Result							
`sllv $s3, $s1, $s2`	$s3	0000	0100	0000	0010	1010	1000	0000	0000
`srlv $s4, $s1, $s2`	$s4	0000	0000	1111	0011	0000	0100	0000	0010
`srav $s5, $s1, $s2`	$s5	1111	1111	1111	0011	0000	0100	0000	0010

Figure Variable-shift operations

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Variable Shift: Why rs and rt seem reversed

- Syntax is sllv rd, rt, rs (and similarly srlv, srav).
- rt contains the value that will be shifted.
- The lower 5 bits of rs give the shift amount.
- rd stores the final result.
- shamt is ignored because the amount now comes from a register.



Easy memory tip: the variable amount comes from a register, not from the instruction bits.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Generating Constants

The addi instruction can be used to assign a **16-bit constant** to a register.

For example, to implement:

int a = 0x4F3C;

MIPS uses:

addi $s0, $0, 0x4F3C

Since $0 always contains zero:

$s0=0+0x4F3C

Code Example 6.10 16-BIT CONSTANT

High-Level Code	MIPS Assembly Code
`int a = 0x4f3c;`	`# $s0 = a` `addi $s0, $0, 0x4f3c  # a = 0x4f3c`

Therefore:
$s0 = 0x00004F3C

For a **32-bit constant**, a single immediate instruction is not sufficient. MIPS uses lui followed by ori.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Generating a 32-bit Constant

To implement:

int a = 0x6D5E4F3C;

MIPS divides the constant into two 16-bit parts:

Upper 16 bits = 0x6D5E

Lower 16 bits = 0x4F3C

First, lui loads the upper half: **lui $s0, 0x6D5E**

giving: $s0 = 0x6D5E0000

Then ori inserts the lower half: **ori $s0, $s0, 0x4F3**

giving: $s0 = 0x6D5E4F3C

Thus:

lui+ori⇒32-bit constant

Code Example 6.11 32-BIT CONSTANT

High-Level Code	MIPS Assembly Code

`int a = 0x6d5e4f3c;`	`# $s0 = a`
	`lui $s0, 0x6d5e  # a = 0x6d5e0000`
	`ori $s0, $s0, 0x4f3c  # a = 0x6d5e4f3c`

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Multiplication and Division

Multiplication and division differ from the other arithmetic operations because they can produce more than one 32-bit result.

Multiplying two 32-bit numbers produces a **64-bit product**. MIPS stores this result in two special-purpose registers:

- hi → upper 32 bits
- lo → lower 32 bits

mult $s0, $s1

For division: div $s0, $s1

the results are:

- lo → quotient
- hi → remainder

The results can be moved to general-purpose registers using: mflo $t0; mfhi $t1

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Multiplication and Division: Numerical

MIPS uses the special **HI** and **LO** registers for multiplication and division. For division, **LO stores the quotient** and **HI stores the remainder**. For example, if: $s0=20,$s1=6

```
div  $s0, $s1
mflo $t0      # $t0 = 3 (quotient)
mfhi $t1      # $t1 = 2 (remainder)
```

because: 20÷6=3 remainder 2

For multiplication, two 32-bit numbers produce a **64-bit product**. HI stores the upper 32 bits and LO stores the lower 32 bits.

```
mult $s0, $s1
mflo $t0      # lower 32 bits
mfhi $t1      # upper 32 bits
```

MIPS also provides mul, which produces a **32-bit result directly in a general-purpose register**:

```
mul $s1, $s2, $s3      # $s1 = $s2 × $s3
```

Remember: div → **LO = quotient, HI = remainder**; mult → **HI:LO = 64-bit product**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Branching

Executing Instructions Out of Sequence

Modern processors improve performance by changing the normal order of execution when a branch instruction occurs.

Types of Branch Instructions:

Conditional Branches

Execution jumps **only if a specified condition is true.**

- **beq (Branch if Equal):** Branches when two registers contain the same value.
- **bne (Branch if Not Equal):** Branches when two registers contain different values.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Branching

Unconditional Branches

Execution always transfers control to another location.

- **j (Jump):** Unconditionally jumps to a specified target address.
- **jr (Jump Register):** Jumps to the address stored in a register (commonly used for function returns).
- **jal (Jump and Link):** Jumps to a function and saves the return address in the **$ra** (return address) register.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Conditional branching (BEQ):

Branch if Equal (beq):

- Compares the values stored in two registers.
- If the values are **equal**, control transfers to the specified **label**.
- If the values are **not equal**, execution continues with the next sequential instruction.

Syntax:

beq $rs, $rt, label

Example Explanation:

- $s0 is initialized to **4**.
- $s1 is initialized to **2**, then doubled to become **4**.
- Since $s0 == $s1, the condition is **true**.
- The processor branches to the label **target**.
- The instructions between beq and target are **skipped**.
- Execution resumes at target, where $s1 becomes **8** (4 + 4).

```
# MIPS assembly
addi   $s0, $0, 4       # $s0 = 0 + 4 = 4
addi $s1, $0, 2         # $s1 = 0 + 2 = 2
add  $s1, $s1, $s1      # $s1 = 2 + 2 = 4
beq  $s0, $s1, target   # branch is taken
addi $s1, $s1, 1        # not executed
sub  $s1, $s1, $s0      # not executed

target:      # label
 add  $s1, $s1, $s0   # $s1 = 4 + 4 = 8
```

Labels indicate instruction location. They can't be reserved words and must be followed by colon (:)

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Conditional branching (BEQ):

Labels:

- A **label** identifies the address of an instruction.
- It acts as the **branch destination**.
- Labels must end with a **colon (:)** and cannot be reserved keywords.

```
# MIPS assembly
addi   $s0, $0, 4       # $s0 = 0 + 4 = 4
addi $s1, $0, 2         # $s1 = 0 + 2 = 2
add  $s1, $s1, $s1      # $s1 = 2 + 2 = 4
beq  $s0, $s1, target   # branch is taken
addi $s1, $s1, 1        # not executed
sub  $s1, $s1, $s0      # not executed

target:      # label
 add  $s1, $s1, $s0   # $s1 = 4 + 4 = 8
```

Labels indicate instruction location. They can't be reserved words and must be followed by colon (:)

Key Takeaway: beq changes the program flow only when the two register values are equal; otherwise, execution proceeds with the next instruction.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

The branch not take (BNE)

Branch if Not Equal (bne):

- Compares the values stored in two registers.
- Branches to the specified **label** only if the register values are **not equal**.
- If the values are **equal**, execution continues with the next sequential instruction.

Syntax:

```
bne $rs, $rt, label
```

Example Explanation

- $s0 is initialized to **4**.
- $s1 is initialized to **2**, then doubled to become **4**.
- Since $s0 == $s1, the condition for bne (**not equal**) is **false**.
- The branch is **not taken**, so execution continues with the next instructions.
- addi updates $s1 from **4 → 5, and subupdates$s1 from **5 → 1.
- Execution eventually reaches the target label, where $s1 becomes **5** (1 + 4).

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

The branch not take (BNE)

```
# MIPS assembly
  addi  $s0, $0, 4          # $s0 = 0 + 4 = 4
  addi  $s1, $0, 2          # $s1 = 0 + 2 = 2
  add   $s1, $s1, $s1       # $s1 = 2 + 2 = 4
  bne   $s0, $s1, target    # branch not taken
  addi  $s1, $s1, 1         # $s1 = 4 + 1 = 5
  sub   $s1, $s1, $s0       # $s1 = 5 – 4 = 1
target:
  add   $s1, $s1, $s0       # $s1 = 1 + 4 = 5
```

Key Takeaway: bne transfers control only when the compared register values are different. If they are equal, execution continues sequentially with the next instruction.

This matches the beq slide in style while clearly highlighting the difference:

- **beq** → Branch when **Equal**.
- **bne** → Branch when **Not Equal**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Unconditional Branching (J)

Jump (j)

- An **unconditional branch** transfers control to another instruction **without checking any condition**.
- The processor always jumps to the specified **target label**.
- Instructions between the j instruction and the target label are **skipped**.

Syntax:

```
j label
```

Example Explanation

- $s0 is initialized to **4**.
- $s1 is initialized to **1**.
- The instruction j target immediately transfers control to the label **target**.
- The instructions sra, addi, and sub are **not executed**.
- Execution resumes at target, where $s1 becomes **5** (1 + 4).

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Unconditional Branching (J)

Applications of j

- Skipping a block of instructions.
- Implementing loops.
- Transferring control to another section of a program.

Key Takeaway: The j instruction always transfers execution to the specified label, regardless of any condition, causing all intermediate instructions to be skipped.

```
# MIPS assembly
  addi $s0, $0, 4          # $s0 = 4
  addi $s1, $0, 1          # $s1 = 1
  j    target              # jump to target
  sra  $s1, $s1, 2         # not executed
  addi $s1, $s1, 1         # not executed
  sub  $s1, $s1, $s0       # not executed

target:
    add  $s1, $s1, $s0     # $s1 = 1 + 4 = 5
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Unconditional Branching (Jr)

Jump Register (jr)

- Transfers control to the **address stored in a register**.
- The target address is determined **at run time**, not encoded in the instruction.
- Unlike j, jr uses a **register operand** instead of a label.
- **jr is an R-type instruction** in the MIPS instruction set.

Syntax:

```
jr $rs
```

Example Explanation

- addi $s0, $0, 0x2010 loads the address **0x00002010** into register $s0.
- jr $s0 immediately jumps to the address stored in $s0.
- The instructions at **0x00002008** and **0x0000200C** are **skipped**.
- Execution resumes at **0x00002010**, where lw $s3, 44($s1) is executed.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Unconditional Branching (Jr)

Applications of jr

- Returning from functions using jr $ra.
- Jumping to addresses computed during program execution.
- Supporting indirect control transfer.

```
# MIPS assembly
0x00002000      addi $s0, $0, 0x2010
0x00002004      jr   $s0
0x00002008      addi $s1, $0, 1
0x0000200C      sra  $s1, $s1, 2
0x00002010      lw   $s3, 44($s1)
```

jr is an **R-type** instruction.

Key Takeaway: jr transfers control to the address contained in a register, enabling indirect jumps and function returns in MIPS programs.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Conditional statements

Conditional statements such as **if**, **if/else**, and **switch/case** execute a block of code only when a specified condition is satisfied.

In MIPS, these high-level constructs are implemented using **conditional branch instructions and labels**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

IF statement

How an if Statement Works

- An if statement executes a block of code **only when a specified condition is true**.
- In MIPS, this is implemented using **conditional branch instructions**.
- The branch instruction checks the **opposite condition** to skip the if block when the condition is false.

Example Explanation

C Code:

```
if (i == j)
    f = g + h;

f = f - i;
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

IF statement

Equivalent MIPS Code:

```
bne $s3, $s4, L1
add $s0, $s1, $s2
L1: sub $s0, $s0, $s3
```

C Code:

```
if (i == j)
   f = g + h;

f = f - i;
```

- bne checks whether i and j are **not equal**.
- If i != j, control jumps to **L1**, skipping f = g + h.
- If i == j, the branch is **not taken**, and f = g + h is executed.
- After the if block, execution continues with f = f - i.

Why use bne instead of beq?

Assembly often tests the **opposite condition** so that it can **skip the if block** when the condition is false, resulting in simpler and more efficient code.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

IF/ELSE statement

How an if-else Statement Works

- An if-else statement executes **one of two blocks** based on a condition.
- In MIPS, this is implemented using a **conditional branch** and an **unconditional jump**.
- The conditional branch selects the appropriate block, while the unconditional jump skips the other block after execution.

Example Explanation

C Code:

```
if (i == j)
   f = g + h;
else
   f = f - i;
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

IF/ELSE statement

Equivalent MIPS Code:

```
bne $s3, $s4, L1
add $s0, $s1, $s2
j done
L1: sub $s0, $s0, $s3
done:
```

C Code:

```
if (i == j)
   f = g + h;
else
   f = f - i;
```

- bne checks whether i and j are **not equal**.
- If i == j, the branch is **not taken**, and f = g + h is executed.
- The j done instruction skips the **else** block.
- If i != j, control branches directly to **L1**, where f = f - i is executed.
- Execution continues from the label **done** after either block.

Key Takeaway: An if-else statement is implemented using a conditional branch to choose the correct block and an unconditional jump to avoid executing the alternate block.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Switch/Case statement

A switch/case selects one of several blocks based on a value.

In MIPS, it can be implemented as a **series of conditional tests**, equivalent to nested if/else statements.

If **none of the cases** match, the **default** block is executed.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Getting Loopy

Loops repeatedly execute a block of code as long as a condition is satisfied.

Common high-level loop constructs are **while** and **for** loops.

In MIPS, loops are implemented using **comparison, branch, and jump instructions**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

While statement

How a while Loop Works

- A while loop repeatedly executes a block of code **as long as the specified condition is true**.
- In MIPS, the loop is implemented using a **conditional branch** to exit the loop and an **unconditional jump** to repeat it.
- The loop condition is evaluated **before each iteration**.

Example Explanation

C Code

```
while (pow != 128) {
    pow = pow * 2;
    x = x + 1;  }
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

While statement

Equivalent MIPS Code

```
while:  beq   $s0, $t0, done
        sll   $s0, $s0, 1
        addi  $s1, $s1, 1
        j     while
done:
```

C Code

```
while (pow != 128) {
   pow = pow * 2;
   x = x + 1;  }
```

- beq checks whether **pow == 128**.
- If the condition is **true**, control branches to **done**, terminating the loop.
- If the condition is **false**, the loop body executes:
  - sll $s0, $s0, 1 multiplies pow by **2** (left shift by 1 bit).
  - addi $s1, $s1, 1 increments x.
- j while returns control to the beginning of the loop to evaluate the condition again.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

While statement

- **Why use beq?**
- The original C condition is **pow != 128**, but the assembly tests the **opposite condition (pow == 128)** to determine when to exit the loop. This simplifies the loop control.

Key Takeaway: A while loop in MIPS uses a conditional branch to exit the loop and an unconditional jump to repeat the loop until the termination condition is met.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

For loop

A **`for` loop** is used to repeatedly execute a block of code when the number of iterations is known or controlled by a loop variable.

Syntax

```
for (initialization; condition; loop operation)
        statement;
```

Components of a `for` Loop

- **Initialization:** Executes **once** before the loop begins.
- **Condition:** Evaluated at the **start of each iteration**. The loop continues while the condition is true.
- **Statement:** The block of code executed during each iteration.
- **Loop Operation (Update):** Executes at the **end of each iteration** before the condition is checked again.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

For loop

Execution Flow

- **Initialization → Condition → Statement → Loop Operation → Condition → … → Exit**

Key Takeaway: A `for` loop combines initialization, condition checking, and update into a single statement, making it ideal for count-controlled iterations.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

For loop in MIPS

How a for Loop Works

- A for loop performs **initialization**, checks a **condition**, executes the **loop body**, updates the loop variable, and repeats until the condition becomes false.
- In MIPS, it is implemented using **initialization instructions**, a **conditional branch**, an **update instruction**, and an **unconditional jump**.

Example Explanation

C Code:

```
for (i = 0; i != 10; i++) {
   sum = sum + i;
}
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

For loop in MIPS

Equivalent MIPS Code:

```
addi $s1, $0, 0      # sum = 0
add  $s0, $0, $0     # i = 0
addi $t0, $0, 10     # loop limit

for:  beq  $s0, $t0, done
      add  $s1, $s1, $s0
      addi $s0, $s0, 1
      j    for
done:
```

- addi instructions initialize **sum**, **i**, and the loop limit (**10**).
- beq checks whether **i == 10**.
- If the condition is **true**, control branches to **done**, terminating the loop.
- Otherwise, the loop body adds the current value of i to sum.
- addi $s0, $s0, 1 increments the loop variable.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

For loop in MIPS

- j for returns control to the beginning of the loop for the next iteration.

Key Takeaway: A for loop in MIPS consists of initialization, condition checking, loop body execution, loop variable update, and an unconditional jump to repeat the loop until the termination condition is met.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Magnitude Comparison Using slt

The beq and bne instructions compare whether two values are **equal or unequal**. For magnitude comparisons, MIPS provides the **Set Less Than (slt)** instruction.

slt $t1, $s0, $s1

The instruction sets $t1 to **1 when $s0 < $s1**, otherwise it sets $t1 to **0**.

For example:

$s0 = 5
$s1 = 8
slt $t1, $s0, $s1

gives:

5<8⇒$t1=1

This can then be combined with beq or bne to implement conditions such as **less than, greater than, less than or equal to, and greater than or equal to**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Loop Using slt : Numerical

The textbook example adds powers of 2 from 1 to 100:

C code:

```
int sum = 0;
for (i = 1; i < 101; i = i * 2)
   sum = sum + i;
```

MIPS equivalent:

```
addi $s1, $0, 0      # sum = 0
addi $s0, $0, 1      # i = 1
addi $t0, $0, 101
loop:
slt  $t1, $s0, $t0   # $t1 = 1 if i < 101
beq  $t1, $0, done   # if i >= 101, exit
add  $s1, $s1, $s0   # sum = sum + i
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Loop Using slt : Numerical

```
sll  $s0, $s0, 1      # i = i * 2
j    loop

done:
```

The values added are:

1+2+4+8+16+32+64=1271+2+4+8+16+32+64=127

Therefore:

sum=127

Key idea: slt converts a magnitude comparison into a **1/0 result**, which can then be tested using a branch instruction.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Arrays and Array Indexing

An array stores multiple elements of the same type in **sequential memory locations**. Each element is identified by an **index**, starting from 0. The address of the first element, array[0], is called the **base address**.

For a 32-bit integer array, each element occupies **4 bytes**. Therefore:

Address of array[i]=Base Address+4i

For example, if the base address is 0x10007000:
array[0] → 0x10007000
array[1] → 0x10007004
array[2] → 0x10007008

To access an element, MIPS uses the base address and an offset:

```
lw $t1, 4($s0)     # loads array[1];  where $s0 contains the base address.
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Accessing an Array in MIPS

Consider: int array[5];

array[0] = array[0] * 8;
array[1] = array[1] * 8;

If the base address is 0x10007000, MIPS first loads this address into $s0:

```
lui $s0, 0x1000
ori $s0, $s0, 0x7000
```

Then the elements are accessed using their byte offsets:

```
lw  $t1, 0($s0)      # array[0]
sll $t1, $t1, 3      # multiply by 8
sw  $t1, 0($s0)
lw  $t1, 4($s0)      # array[1]
sll $t1, $t1, 3      # multiply by 8
sw  $t1, 4($s0)
```

Since each integer occupies **4 bytes**, array[1] is located **4 bytes after** array[0].

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Array Access Using a for Loop

For a large array, the same instructions should not be repeated for every element. The index can be used to calculate the address of each element.

For a 1000-element integer array:

```
for (i = 0; i < 1000; i++)
    array[i] = array[i] * 8;
```

MIPS calculates the address using:

```
sll $t0, $s1, 2       # $t0 = i × 4
add $t0, $t0, $s0     # address of array[i]
lw  $t1, 0($t0)
sll $t1, $t1, 3       # array[i] × 8
sw  $t1, 0($t0)
```

Here, $s0 contains the **base address** and $s1 contains i.

Byte offset=i×4

The shift sll $t0,$s1,2 is used because: i×4=i×2^2

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Byte Arrays and Characters

Characters can be stored in a **single byte** instead of a 32-bit word. MIPS provides special instructions for accessing byte data:

```
lbu    # Load Byte Unsigned
lb     # Load Byte
sb     # Store Byte
```

lbu **zero-extends** the loaded byte to 32 bits, while lb **sign-extends** it. sb stores only the **least significant byte** of a register into memory.

For byte arrays, consecutive elements are only **1 byte apart**:

Address of chararray[i]=Base Address+i

This differs from an integer array, where the offset is **4i.**

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Character Array: Lowercase to Uppercase Numerical

Question:

Convert the following C code into MIPS assembly language:

```
char chararray[10];
int i;
for (i = 0; i != 10; i = i + 1)
    chararray[i] = chararray[i] - 32;
```

Assume that $s0 already contains the base address of chararray.

Solution

```
addi $s1, $0, 0        # i = 0
addi $t0, $0, 10       # $t0 = 10
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Character Array Numerical

```
loop:
beq  $t0, $s1, done      # if i == 10, exit
add  $t1, $s1, $s0       # address of chararray[i]
lb   $t2, 0($t1)         # load chararray[i]
addi $t2, $t2, -32       # convert lowercase to uppercase
sb   $t2, 0($t1)         # store result
addi $s1, $s1, 1         # i = i + 1
j    loop
done:
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Character Array Numerical

Why add $t1, $s1, $s0?

Each character occupies **1 byte**, so the address of chararray[i] is:

Base Address+i

For ASCII characters:
′a′=0x61, 0x61−0x20=0x41=′A′

Thus, subtracting **32 (0x20)** converts a lowercase English letter to its corresponding uppercase letter.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Function Calls and Returns

Functions allow frequently used code to be reused, making programs more **modular and readable**. A function receives **arguments** and produces a **return value**.

MIPS follows a convention for communication between the caller and callee:

$a0–$a3→input arguments; $v0–$v1→return value

The function is called using jal, which stores the return address in $ra and jumps to the function. The function returns using jr $ra.

```
jal simple      # call function
...
simple:
jr $ra          # return to caller
```

For example, if jal is at 0x00400200, the next instruction is at 0x00400204. Therefore: $ra=0x00400204\boxed{\ $ra=0x00400204}

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Function Call with Arguments

Consider the function:

```
int diffofsums(int f, int g, int h, int i) {
    int result;
    result = (f + g) - (h + i);
    return result;
}
```

The caller places the four arguments in $a0–$a3:

```
addi $a0, $0, 2
addi $a1, $0, 3
addi $a2, $0, 4
addi $a3, $0, 5
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Function Call with Arguments

jal diffofsums

add $s0, $v0, $0

The callee performs the calculation and places the result in $v0:

add $t0, $a0, $a1

add $t1, $a2, $a3

sub $s0, $t0, $t1

add $v0, $s0, $0

jr $ra

Numerically:

(2+3)−(4+5)=5−9=−4(2+3)-(4+5)=5-9=\boxed{-4}

Therefore, the function returns **−4 in $v0**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Stack in Function Calls

The **stack** is a LIFO memory structure used to store temporary data and local variables during function execution. The MIPS stack grows **toward lower memory addresses**.

$sp is the **stack pointer** and points to the top of the stack.

If: $sp=0x7FFFFFFC and a function needs two additional words:

addi $sp, $sp, -8

then: $sp=0x7FFFFFF4

because each word occupies **4 bytes**.

A function follows five basic steps when saving registers:

1. Allocate stack space.
2. Store registers on the stack.
3. Execute the function.
4. Restore the registers.
5. Deallocate the stack space.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Saving Registers on the Stack

A function must preserve registers that the caller depends on. The stack can temporarily store these registers while the function executes.

For diffofsums, three registers are saved:

```
addi $sp, $sp, -12     # space for 3 words
sw   $s0, 8($sp)
sw   $t0, 4($sp)
sw   $t1, 0($sp)
# function operations
lw   $t1, 0($sp)
lw   $t0, 4($sp)
lw   $s0, 8($sp)
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Saving Registers on the Stack

```
addi $sp, $sp, 12

jr   $ra
```

Since each register requires one word: 3×4=12 bytes

The stack space allocated by a function is called its **stack frame**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Preserved and Nonpreserved Registers

MIPS divides registers into **preserved** and **nonpreserved** registers.

Preserved	Nonpreserved
$s0–$s7	$t0–$t9
$ra	$a0–$a3
$sp	$v0–$v1

A **callee must save and restore** preserved registers that it modifies. Nonpreserved registers may be changed by the callee.

Therefore, in the improved diffofsums, only $s0 needs to be saved:

```
addi $sp, $sp, -4
sw   $s0, 0($sp)

# function operations
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Preserved and Nonpreserved Registers

```
lw   $s0, 0($sp)
addi $sp, $sp, 4
jr   $ra
```

Thus, the stack frame requires only:

1×4=4 bytes

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Recursive Function Calls

A **leaf function** does not call another function, whereas a **nonleaf function** calls another function. A recursive function is a nonleaf function that **calls itself**.

Consider factorial:

```
int factorial(int n) {
   if (n <= 1)
      return 1;
   else
      return n * factorial(n - 1); }
```

The recursive relationship is: $factorial(n)=n \times factorial(n-1)$

with the base case: $factorial(1)=1$

For n=3: $3!=3 \times 2 \times 1=6$

Each recursive call creates its own stack frame to preserve $a0 and $ra.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Recursive Factorial in MIPS

The textbook implementation saves $a0 and $ra before making the recursive call:

```
factorial:
addi $sp, $sp, -8
sw   $a0, 4($sp)
sw   $ra, 0($sp)

addi $t0, $0, 2
slt  $t0, $a0, $t0
beq  $t0, $0, else

addi $v0, $0, 1
addi $sp, $sp, 8
jr   $ra

else:
addi $a0, $a0, -1
jal  factorial
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Recursive Factorial in MIPS

```
lw   $ra, 0($sp)
lw   $a0, 4($sp)
addi $sp, $sp, 8

mul  $v0, $a0, $v0
jr   $ra
```

For factorial(3):

3→2→1

Then the calls return:

1→2×1=2→3×2=6

Each invocation allocates **8 bytes** for $a0 and $ra.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

Additional Arguments and Local Variables

MIPS provides registers $a0–$a3 for the first **four input arguments**. If a function requires more than four arguments, the additional arguments are passed through the **stack**.

First 4 arguments → $a0–$a3
Additional arguments → Stack

The caller expands its stack to make room for the additional arguments. A function may also use the stack for **local variables and local arrays**.

The callee's stack frame can contain:

Local variables / arrays
$s0–$s7 (if needed)
$a0–$a3 (if needed)
Return address
Additional arguments

An important exception is that when a function has more than four arguments, the callee may access the **additional arguments in the caller's stack frame.Key MIPS Function-Call Flow:** Arguments→jal→Callee→Return value→jr $ra

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

Think About It

1. Assume that register $s1 contains 7 and register $s2 contains 4. Consider the following instruction sequence (the first register is the destination, and the not instruction inverts each bit):

```
nor  $s3, $s0, $s2
addi $s3, $s2, 1
add $s3, $s1, $s2
```

1. What value gets stored in $s3? What arithmetic operation do above instructions perform on $s1 and $s2 contents?

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

Q1. What is the primary role of the Program Counter (PC)?

A) Store arithmetic results

B) Hold the address of the next instruction to be executed

C) Store data values temporarily

D) Decode machine instructions

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

MCQ

Q2. Which statement correctly distinguishes assembly code from machine code?

A) Assembly code is executed directly by the CPU.

B) Machine code uses mnemonics such as add and lw.

C) Assembly code is human-readable and must be translated into machine code.

D) Both are identical representations of a program.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

Q3. Which MIPS instruction transfers control only when two register values are equal?

A) bne

B) j

C) beq

D) jr

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

Q4. Which instruction is commonly used to return from a function in MIPS?

A) beq

B) j

C) jal

D) jr

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

Q5. In the MIPS implementation of an if-else statement, why is the instruction j done used?

A) To initialize variables before execution

B) To skip the else block after executing the if block

C) To compare two registers

D) To terminate the program

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

Q6. In a MIPS while loop, which instruction causes the loop to repeat?

A) beq

B) addi

C) j while

D) sub

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

Q7. Which sequence correctly represents the execution order of a for loop?

A) Statement → Initialization → Condition → Update

B) Initialization → Condition → Statement → Update

C) Condition → Initialization → Update → Statement

D) Initialization → Statement → Update → Condition

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

Q8. An integer array has base address 0x10007000. What address does the following MIPS code access?

Assume $s0 = 0x10007000.

A. 0x10007006

B. 0x10007018

C. 0x10007024

D. 0x10007028

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

Q9. Consider the following function call:

```
addi $a0, $0, 2

addi $a1, $0, 3

addi $a2, $0, 4

addi $a3, $0, 5
jal  diffofsums
```

where the function computes: result = (f + g) - (h + i);

What value should be returned in $v0?

A. -4
B. 0
C. 4
D. 14

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC AND COMPUTER DESIGN

MCQ

ANSWER KEY:

Q1: B **Q2: C** **Q3: C** **Q4: D** **Q5: B** **Q6: C** **Q7: B** **Q8: B** **Q9: A**

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.



THANK YOU

Team DLCD

Department of CSE and CSE(AI & ML)